

Министерство науки и высшего образования РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОМСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра «Математические методы и информационные технологии
в экономике»

Отчет по лабораторной работе №8

По дисциплине
«Операционные системы»

Выполнил:

Студент группы ПИ-221

Нурсалимов Д. Е.

Проверил: ст. преподаватель
Карабцов Р.Д.

Омск 2023

Цель работы. Изучение методов построения многопоточных программ с синхронизацией.

Задание. Разработать многопоточную программу, отображающую на экране взаимодействие трех нитей «читателей» из общей области данных и трех «писателей», записывающих в этот буфер данные. Буфер предназначен для хранения 12 символов. Первая нить-писатель выводит латинскими буквами название города "Novosibirsk", вторая нить-писатель выводит латинскими буквами название города "Semipalatinsk", а третья — название города "Ekaterinburg". Такой вывод эти три нити осуществляют в два приема, первый из которых записывает половину своего текста без завершающего этот промежуточный текст нуля. Между такими половинами вывода нити производят задержку на случайную величину миллисекунд, но не более 1 сек. После вывода своего текста в буфер каждая нить-писатель переходит в ожидание порядка 2-3 сек до следующей попытки записи в буфер. Нити-читатели - через случайный интервал порядка 300 мсек - читают данные из буфера, если это позволяют средства синхронизации доступа между нитями, и вывод прочитанный текст на экран, каждая в свой столбец. Каждый вывод нити-читателя осуществляется в новую строку своего столбца, поэтому суммарные действия вывода в таких нитях предусмотреть только для 20 - 24 строк. Синхронизацию осуществить с помощью семафоров.

Код программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <process.h>
#include <conio.h>
HANDLE consoleout; // хэндлы
HANDLE Semaphore, Semaphore2;
CRITICAL_SECTION CriticalSection; // критическая секция - обеспечивает синхронизацию
int delay1, delay2; // задержки
char Buffer[12]; // буфер для записи
char names[3][14] = { // двумерный массив с городами
    {"Novosibirsk"},
    {"Semipalatinsk"},
    {"Ekaterinburg"}};
// счетчик элементов в массиве
int n = 0;
// задает размер консольного окна
void SetWindow(int Width, int Height)
{
    COORD coord;
```

```

coord.X = Width;
coord.Y = Height;

SMALL_RECT Rect;
Rect.Top = 0;
Rect.Left = 0;
Rect.Bottom = Height - 1;
Rect.Right = Width - 1;

SetConsoleScreenBufferSize(consoleout, coord); // передает размер,
устанавливая буфер консоли
SetConsoleWindowInfo(consoleout, TRUE, &Rect); // передает информацию
окну
}
// поток функция записи
DWORD WINAPI Write(void *arg)
{
    int r = (int)arg; // получение аргумента и запись в новую переменную
    r--; // для получения элемента массива, уменьшение числа
    // вход в бесконечный цикл
    while (1)
    {
        WaitForSingleObject(Semaphore, INFINITE); // ожидание семафора 1
        // перебор первых 6 символов
        for (n = 0; n < 6; n++)
        {
            Buffer[n] = names[r][n];
        }
        Sleep(100 * (int)delay1); // задержка
        // перебираем остальные символы и записываем в буфер
        for (n = 6; n < 12; n++)
        {
            Buffer[n] = names[r][n];
        }
        ReleaseSemaphore(Semaphore2, 1, NULL); // освобождение семафора 2
        Sleep(2000 + rand() % 1001); // задержка
    }
}
// функция чтения
DWORD WINAPI Read(void *arg)
{
    COORD pos; // позиция курсора
    char buff[12]; // буфер
    int lenBuff; // переменная для хранения количества записанного в буфер
текста
    for (int i = 0; i < 40; i++)
    {
        WaitForSingleObject(Semaphore2, INFINITE); // ожидание семафора 2

```

```

pos.X = 20 * (int)arg;    // относительно аргумента меняется позиция текста
pos.Y = i + 1;
EnterCriticalSection(&CriticalSection);    // вход в критическую зону
SetConsoleCursorPosition(consoleout, pos);    // установка курсора в консоли
// установка цвета относительно переданного аргумента
switch((int)arg){
    case 1: SetConsoleTextAttribute(consoleout, FOREGROUND_RED |
FOREGROUND_BLUE | FOREGROUND_INTENSITY);
        break;
    case 2: SetConsoleTextAttribute(consoleout, FOREGROUND_GREEN |
FOREGROUND_INTENSITY);
        break;
    case 3: SetConsoleTextAttribute(consoleout, FOREGROUND_RED |
FOREGROUND_INTENSITY);
        break;
}
lenBuff = sprintf(buff, "%s", Buffer);    // запись в буфер, возвращает значение
lenBuff = ((int)arg > 1) ? lenBuff - 1 : lenBuff;    // для того чтобы лишний
символ не был записан
WriteFile(consoleout, buff, lenBuff, NULL, NULL);    // вывод в консоль
LeaveCriticalSection(&CriticalSection);    // выход из критической секции
ReleaseSemaphore(Semaphore, 1, NULL);    // освобождение семафора 1
Sleep(100 * (int)delay2);    // задержка
}
return 0;
}

```

```

void main(int args, char *argv[])
{
    system("cls");    // очистка консоли
    consoleout = GetStdHandle(STD_ERROR_HANDLE);    // получение хэнгла
консоли
    SetWindow(120, 30);    // установка размеров консоли
    HANDLE writeThread[3];    // хэндлы потоков
    HANDLE readThread[3];
    delay1 = 20;    // значения задержок по умолчанию
    delay2 = 3;
    InitializeCriticalSection(&CriticalSection);    // инициализация критической
зоны
    // если аргументы в наличии, запись в переменные
    if (args >= 3)
    {
        sscanf(argv[1], "%d", &delay1);
        sscanf(argv[2], "%d", &delay2);
    }
    Semaphore = CreateSemaphore(NULL, 1, 1, NULL);    // инициализация
семафора 1
    Semaphore2 = CreateSemaphore(NULL, 0, 1, NULL);    // инициализация

```

семафора 2

```
// запуск потоков
for (int i = 0; i < 3; i++)
{
    writeThread[i] = CreateThread(NULL, 4096, Write, (void *) (i + 1), 0, NULL);
    readThread[i] = CreateThread(NULL, 4096, Read, (void *) (i + 1), 0, NULL);
}
getchar(); // ожидания ввода
DeleteCriticalSection(&CriticalSection); // удаление секции
// завершение потоков
for (int i = 0; i < 3; i++)
{
    CloseHandle(writeThread[i]);
    CloseHandle(readThread[i]);
}
}
```

Результат выполнения:

Novosibirsk	Semipalatinsk	Ekarelinburg
Novosibirsk	Semipalatinsk	Ekarelinburg
Novosibirsk	Semipalatinsk	Ekarelinburg
Novosibirsk	Semipalatinsk	Ekarelinburg
Novosibirsk	Semipalatinsk	Ekarelinburg
Novosibirsk	Semipalatinsk	Ekarelinburg
Novosibirsk	Semipalatinsk	Ekarelinburg

Рис. 1 –Выполнение программы.